

Visione Artificiale

Assignment 2

Salvo Portuesi | salvo.portuesi@community.unipa.it

Architettura del sistema

Il sistema è composto da tre moduli principali, i quali si dividono le responsabilità nel modo seguente:

- A. Preprocessing delle immagini, estrazione dei descrittori locali, calcolo degli istogrammi di visual words e classificazione di un'immagine fissato un modello pre-addestrato e un vocabolario di visual words;
- B. Calcolo e salvataggio dei possibili vocabolari di visual words attraverso il clustering al variare degli iper-parametri;
- C. Addestramento, valutazione e salvataggio dei modelli di classificazione al variare degli iper-parametri.

Gli ultimi due moduli menzionati sono necessari solo durante lo sviluppo e l'addestramento.

Modulo A: componenti di base e inferenza

Estrattore di descrittori locali:

- Data un'immagine, viene restituita una matrice in cui ogni riga rappresenta un descrittore di un keypoint.
- Ha come parametri un metodo per ottenere i descrittori (SIFT o ORB), un'eventuale normalizzazione per i descrittori (L1 o L2) e un numero di descrittori target (utile per limitare le risorse necessarie nello sviluppo).

Costruttore di istogrammi di visual word:

- Data un'immagine, essa viene convertita in scala di grigi e vengono estratti i suoi descrittori locali. Ogni descrittore viene assegnato al centroide del vocabolario di visual word più vicino e ne viene calcolato l'istogramma.
- Ha come parametri l'estrattore dei descrittori locali, il vocabolario di visual word e un'eventuale normalizzazione per gli istogrammi (L1 o L2).

Inferenza:

- Dato il path di un'immagine, quest'ultima viene caricata e ne viene calcolato un istogramma rappresentativo fissato un vocabolario di visual words. Viene poi eseguita la classificazione fissato un modello pre-addestrato.

Modulo B: calcolo dei vocabolari di visual words

- Vengono calcolati 18 vocabolari di visual words al variare dei parametri dell'estrattore di descrittori locali e del numero di cluster sfruttando il parallelismo a livello di processo per migliorare le prestazioni.
- Le immagini vengono caricate dal dataset AID senza considerare le label associate.
- Le visual words vengono individuate applicando il clustering ai descrittori individuati per ogni immagine pre-processata.
- In particolare, viene utilizzata la variante batch di K-means, la quale permette di individuare i centroidi senza avere a disposizione tutti i descrittori contemporaneamente. Si è scelto di utilizzare una singola iterazione dei batch a causa dell'alta disponibilità di descrittori (> 10,000,000) e si è verificato che, anche se il numero di iterazioni viene posto a tre, l'accuratezza del classificatore migliore non subisce miglioramenti significativi.
- Il calcolo ha richiesto circa 40 minuti sul processore AMD Ryzen 7 4800HS producendo circa 1.5 MB di dati.

Modulo C: addestramento dei classificatori

- Vengono calcolati 270 modelli di classificatori al variare dei vocabolari di visual words, dei tipi di classificatore e dell'eventuale normalizzazione per gli istogrammi.
- Le immagini vengono caricate dal dataset UC Merced Land Use.
- Nella valutazione dei modelli viene utilizzato un 3-fold cross-validation protocol, ma viene eseguito un ultimo addestramento sull'intero dataset per calcolare il modello finale da salvare.
- Poiché in un generico dataset il numero di immagini per classe potrebbe avere una certa variabilità, i modelli di classificazione usano dei pesi appropriati per ogni classe. In realtà, il dataset usato non è caratterizzato da tale variabilità, pertanto i pesi sono superflui.
- Il calcolo ha richiesto circa 5 minuti sul processore AMD Ryzen 7 4800HS producendo circa 2.3 GB di dati.

Problemi riscontrati durante lo sviluppo

- In alcune immagini del dataset AID non viene trovato nessun descrittore, quindi si è scelto di ignorare tali casi nel calcolo dei vocabolari. Inoltre, si è gestito tale caso nell'estrattore di descrittori locali e nel costruttore di istogrammi di visual word al fine di non applicare la normalizzazione sul vettore nullo.
- Per garantire l'indipendenza del *Modulo A* usato per l'inferenza, è stato necessario risolvere un problema di importazione circolare e scegliere accuratamente la collocazione di ogni funzione e classe.
- Nel salvataggio dei classificatori e dei vocabolari la directory specifica potrebbe non esistere, pertanto è necessario crearla.
- Nel calcolo delle metriche di valutazione dei classificatori potrebbero comparire dei denominatori nulli, quindi si è specificato un valore sostitutivo pari a zero.

Risultati ottenuti in cross-validation e modello selezionato

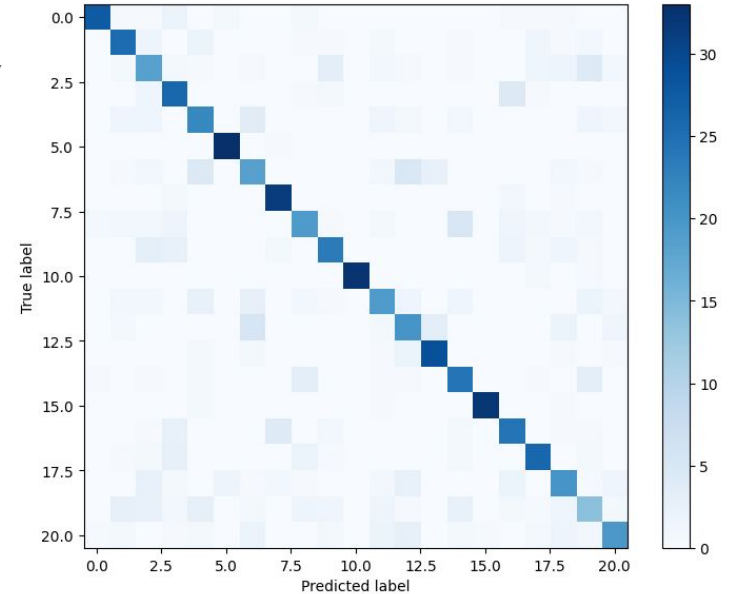
La seguente tabella descrive i risultati ottenuti da ogni tipologia di classificatore nella configurazione di iper-parametri con accuratezza più alta. Le metriche sono riportate come media sui tre fold.

Classifier	BoW Method	BoW Norm	Bow Clusters	Hist Norm	Accuracy	Precision	Recall	F1-score
SVM RBF	SIFT	None	500	L2	0.7223	0.7270	0.7224	0.7209
SVM Linear	SIFT	None	500	L2	0.7142	0.7170	0.7143	0.7113
Logistic Regression	SIFT	L1	500	L2	0.6946	0.6967	0.6948	0.6866
Random Forest	SIFT	L2	100	None	0.6526	0.6542	0.6530	0.6476
k-NN	SIFT	L2	100	None	0.6264	0.6373	0.6267	0.6203

Risultati ottenuti in cross-validation e modello selezionato

Dalle metriche ottenute per ogni possibile configurazione, si è visto che l'accuratezza del modello rimane stabile al variare del parametro *BoW Norm*, pertanto si potrebbe rimuovere la normalizzazione dei descrittori per ciascun tipo di classificatore al fine di risparmiare risorse computazionali nell'inferenza.

Di fianco si trova la matrice di confusione relativa al modello migliore (SVM con kernel RBF), la quale è stata ottenuta calcolando la media delle singole matrici nei tre fold. Per motivi grafici si è scelto di non rappresentare le label categoriche.

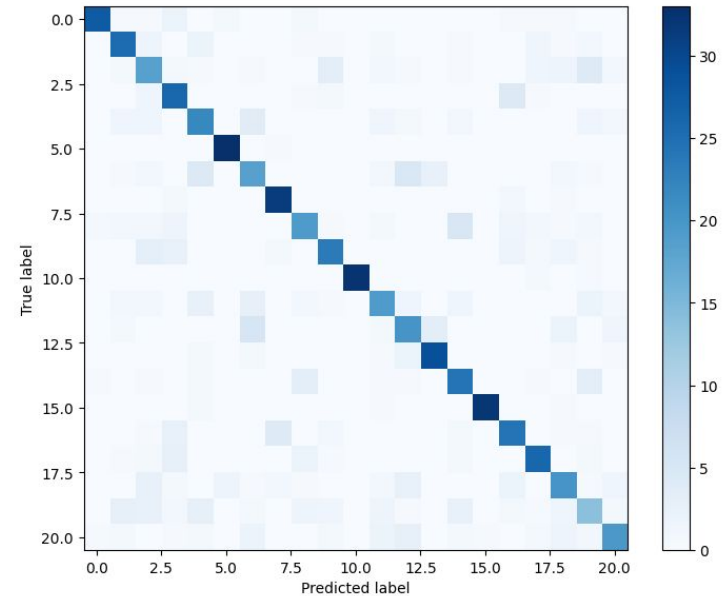


Risultati ottenuti in cross-validation e modello selezionato

Si è visto che in molti casi le metriche calcolate (Accuracy, Macro Precision, Macro Recall e Macro F1-score) hanno tutti valori molto simili se valutate per una singola configurazione.

Analiticamente si dimostra che le metriche sono esattamente uguali quando le classi sono perfettamente bilanciate e la matrice di confusione è simmetrica ($FP_i = FN_i$).

Pertanto, nel caso in esame la similitudine è coerente perché le classi sono bilanciate e in molte configurazioni la matrice di confusione è pressoché simmetrica.



Possibili miglioramenti del sistema

- Il calcolo dei vocabolari di visual words avviene in modo indipendente tra le tre possibili configurazioni di normalizzazione dei descrittori (L1, L2 e assenza di normalizzazione), ma ciò è uno spreco di risorse perché, fissato un metodo (SIFT o ORB), i descrittori iniziali sono gli stessi. Vale un ragionamento analogo per la normalizzazione degli istogrammi. Si noti che implementare tale miglioria è complicato a causa della parallelizzazione e della suddivisione in batch.
- I modelli di classificazione utilizzano gli iper-parametri di default e, talvolta, alcuni di questi non convergono. Pertanto, si potrebbe selezionare un insieme di configurazioni *più promettenti* su cui ottimizzare la scelta degli iper-parametri individualmente.
- I descrittori SIFT e ORB possono essere estratti solo da un singolo canale dell'immagine, pertanto l'informazione di colore non viene considerata dal classificatore. È quindi possibile usare una concatenazioni di istogrammi calcolati sui singoli canali di uno spazio colore.
- Invece di usare un singolo classificatore, si potrebbe costruire un'ensemble di classificatori combinando più modelli per ottenere una previsione più accurata.

Sitografia e tool utilizzati

- Documentazione NumPy: [save](#), [norm](#), eccetera.
- Documentazione Scikit-learn: [StratifiedKFold](#), [confusion_matrix](#), eccetera.
- Gemini: [Metric Equality Conditions Explained](#).
- GPT-5.2-Codex: per l'implementazione della funzione `parse_args()`; per la scrittura del file *README.md*; per individuare un metodo di salvataggio dei modelli di classificatore e per l'implementazione iniziale del parallelismo a livello di processo.
- Autocompletamento del codice in Visual Studio Code.